Aide-mémoire PySpark et SQL

Banque Lumen (fictive). Remplacez `<stockage>` par le storage account de la session et `lumen_stagiaire01` par votre catalog. En tête de notebook : `from pyspark.sql import functions as F`. En SQL, une cellule commence par `%sql`.

Chemin des fichiers bruts : `abfss://formation@<stockage>.dfs.core.windows.net/banque/`, noté `<banque>` ci-dessous.

Lire

PySpark	SQL
`spark.read.format("csv").option("header", True).option("sep", ";").load("<banque>/clients.csv")`	`SELECT * FROM read_files('<banque>/clients.csv', format => 'csv', header => true, sep => ';')`
`spark.read.parquet("<banque>/transactions/")` (tout le dossier)	`SELECT * FROM read_files('<banque>/transactions/', format => 'parquet')`
`spark.table("lumen_stagiaire01.bronze.transactions")`	`SELECT * FROM lumen_stagiaire01.bronze.transactions`

Un CSV lu par `spark.read` est tout en texte, sauf `.option("inferSchema", True)` ; `read_files` devine les types.

Transformer

PySpark	SQL
`df.select("compte_id", "montant")`	`SELECT compte_id, montant`
`df.filter(F.col("devise") == "EUR")` ou `.where(...)`	`WHERE devise = 'EUR'`
`df.filter((F.col("montant") < 0) & (F.col("pays") != "FR"))`	`WHERE montant < 0 AND pays <> 'FR'`
`df.withColumn("jour", F.col("horodatage").cast("date"))`	`CAST(horodatage AS DATE) AS jour`
`df.withColumn("source", F.lit("bronze"))`	`'bronze' AS source`
`df.withColumn("debit", F.abs("montant"))`	`abs(montant) AS debit`
`F.to_date("date_entree", "dd/MM/yyyy")`	`to_date(date_entree, 'dd/MM/yyyy')`
`df.withColumnRenamed("libelle", "label")`	`libelle AS label`
`df.drop("libelle")`	`SELECT * EXCEPT (libelle)`
`df.dropDuplicates()`	`SELECT DISTINCT ...`

Agréger

PySpark	SQL
`df.groupBy("categorie").agg(F.count("*").alias("operations"), F.sum("montant").alias("total"), F.avg("montant").alias("moyenne"))`	`SELECT categorie, COUNT(*) AS operations, SUM(montant) AS total, AVG(montant) AS moyenne ... GROUP BY categorie`
`df.groupBy("segment").count()`	`SELECT segment, COUNT(*) AS count ... GROUP BY segment`
`df.orderBy(F.desc("total"))`	`ORDER BY total DESC`
`df.limit(10)`	`LIMIT 10`

Joindre

PySpark	SQL
`tx.join(co, "compte_id")` (inner, par défaut)	`FROM tx JOIN co USING (compte_id)`
`tx.join(co, "compte_id", "left")` (garde tout `tx`)	`FROM tx LEFT JOIN co USING (compte_id)`
`tx.join(co, "compte_id", "left_anti")` (tx sans compte)	`FROM tx LEFT ANTI JOIN co USING (compte_id)`
`tx.join(co, tx.compte_id == co.compte_id)`	`FROM tx t JOIN co c ON t.compte_id = c.compte_id`

Une jointure inner écarte sans prévenir les lignes sans correspondance : comptez avant et après.

Écrire

PySpark	SQL
`df.write.mode("overwrite").saveAsTable("lumen_stagiaire01.silver.x")` (remplace)	`CREATE OR REPLACE TABLE lumen_stagiaire01.silver.x AS SELECT ...`
`df.write.mode("append").saveAsTable("lumen_stagiaire01.silver.x")` (ajoute)	`INSERT INTO lumen_stagiaire01.silver.x SELECT ...`
`df.write.saveAsTable(...)` (mode `error`, par défaut : échoue si la table existe)	`CREATE TABLE lumen_stagiaire01.silver.x AS SELECT ...`
`.option("overwriteSchema", True)` (le schema change en `overwrite`)	`CREATE OR REPLACE TABLE` remplace aussi le schema
`df.createOrReplaceTempView("v")` (vue de session, lisible en SQL) ; `spark.sql("SELECT ... FROM v")`	`CREATE OR REPLACE TEMP VIEW v AS SELECT ...`

`saveAsTable` crée une table Delta managed, le format par défaut de Databricks.

Delta utile

PySpark	SQL
`display(spark.sql("DESCRIBE HISTORY lumen_stagiaire01.silver.x"))`	`DESCRIBE HISTORY lumen_stagiaire01.silver.x`
`spark.read.option("versionAsOf", 1).table("lumen_stagiaire01.silver.x")`	`SELECT * FROM lumen_stagiaire01.silver.x VERSION AS OF 1`
`spark.read.option("timestampAsOf", "2026-11-17").table("lumen_stagiaire01.silver.x")`	`SELECT * FROM lumen_stagiaire01.silver.x TIMESTAMP AS OF '2026-11-17'`
`display(spark.sql("DESCRIBE DETAIL lumen_stagiaire01.silver.x"))`	`DESCRIBE DETAIL lumen_stagiaire01.silver.x` (format, emplacement, fichiers)

Afficher et inspecter

PySpark	SQL
`display(df)` (tableau, graphique, data profile)	le résultat de la cellule `%sql`
`df.printSchema()`	`DESCRIBE TABLE lumen_stagiaire01.bronze.clients`
`df.count()`	`SELECT COUNT(*) FROM ...`
`df.show(5)` (texte brut)	`SELECT * FROM ... LIMIT 5`
`df.columns`, `df.dtypes`	`SHOW COLUMNS IN lumen_stagiaire01.bronze.clients`
`display(dbutils.fs.ls("<banque>"))`	`LIST '<banque>'`

Rappel : les transformations (`select`, `filter`, `join`...) ne calculent rien ; une action (`count`, `display`, `write`) lance le calcul.